

Sushi — Game Design Document

Game: Sushi, a counter-service sushi restaurant sim

Platform: iPad and Mac (Xogot / Godot 4.7, Mobile renderer). Touch first; mouse works identically (one pointer).

Engine stamp: `config/features = ["4.7", "Mobile"]` (Xogot 1.7.2 / Godot 4.7.2) — never upgraded by the build.

Art: Quaternius *Sushi Restaurant Kit* (May 2023) and Quaternius *Ultimate Food Pack* (Oct 2019), both CC0, imported at `res://assets/Sushi Restaurant Kit - May 2023/` and `res://assets/Ultimate Food Pack - Oct 2019/` (the packs' own folder names, structure untouched).

Document version: 1.0 — 2026-09-08

Status: Specification. At the time of writing the project contains the two asset packs, this document and nothing else: zero scenes, zero scripts.

Contents

0. How to read this document

1. Concept

2. Design pillars

Pillar 1 — Everything is one tap

Pillar 2 — Read the counter, not the HUD

Pillar 3 — Three minutes, one more day

3. Core gameplay loop

3.1 One order, second by second

3.2 What makes it repeatable

4. Controls

5. Camera and perspective

6. Systems

6.1 Restaurant layout

6.2 Stations and the plate

6.3 Customers

6.4 The chef

6.5 Scoring

6.6 Day rating

6.7 Fail and win states

6.8 Persistence

7. Scene and node architecture

8. Art direction

9. Audio plan

10. UI and HUD

10.1 Main menu

10.2 Day select

10.3 HUD (in a day)

10.4 Pause

10.5 Results

10.6 Ending

11. Content plan: the five days

12. Scope

12.1 Cut list — deliberately OUT of v1

12.2 Build order (each step ends with a run in Xogot, a screenshot check, and a commit)

13. Missing assets and open questions

0. How to read this document

This is the build specification for an implementing agent. Every 3D object named in this document is a real file in one of the two packs; the reference form is the model's base name, for example `Environment_Counter_Straight` means `res://assets/Sushi Restaurant Kit - May 2023/Environment/glTF/Environment_Counter_Straight.glTF`. Names prefixed **UFP:** come from the Ultimate Food Pack's `FBX/` folder (for example **UFP:** `Egg_Fried` is `res://assets/Ultimeate Food Pack - Oct 2019/FBX/Egg_Fried.fbx`). Anything the design needs that neither pack contains is listed in section 13 and the design works without it.

Units. One Godot unit is one "pack unit". The kit is authored on a 4-unit tile: every floor tile and wall segment is 4×4 , a counter segment is 4 wide $\times$ 1.92 tall $\times$ 1.76 deep, a stool is 0.88 tall, a rabbit customer stands 4.16 tall in its T-pose file (2.9 tall at the 0.7 scale used in game). All positions below are in these units on a grid whose origin is the centre of the counter's front edge, X to the right, Z toward the camera.

Runtime rules. Every scene is a saved `.tscn` built in the Xogot editor with semantic node names. Scripts only add behaviour; they never construct the scene tree. Customers, dishes and ingredients are instanced from saved prefab scenes.

Terminology. *Station* — a tappable ingredient source on the counter. *Plate* — the assembly area where the current dish is built. *Order* — one dish requested by one customer. *Day* — one run of the core loop; the game has 5 days.

1. Concept

You are Panda, the chef of a four-stool sushi bar. Rabbit customers walk in, sit at the counter and show what they want; you tap ingredients to build the dish on your plate, then tap the customer to serve it before their patience runs out. Serve fast for tips, keep the streak alive, and survive five increasingly busy days.

Session length: one day lasts 2:30–3:30 minutes. A complete first playthrough of the five days is 15–20 minutes. Days can be replayed for a better star rating.

2. Design pillars

Pillar 1 — Everything is one tap

Ingredients, plate, customers and the bin are all single taps on 3D objects you can see. There are no drags, no holds, no menus during play.

Test: hand the iPad to someone who has never seen the game; if they serve their first correct dish within 40 seconds of Day 1 starting, the pillar holds.

Pillar 2 — Read the counter, not the HUD

An order is shown as the real dish model floating over the customer's head; the plate shows the real ingredients you have added; patience is a ring around the order. A player can play with the HUD hidden.

Test: hide the HUD layer and play Day 2; if a tester can still complete it with ≥ 2 stars, the pillar holds.

Pillar 3 — Three minutes, one more day

A day is short enough to replay on impulse and long enough to build a streak. Difficulty rises by a fixed schedule (section 11), never by randomness alone.

Test: the timer on the results screen. Any day whose median tester time exceeds 3:45 gets customers removed, not sped up.

3. Core gameplay loop

3.1 One order, second by second

Time	What happens
0.0 s	A rabbit enters through the door at the left, walks to a free stool (2.0 s walk), sits. The order bubble appears: the dish model spinning above its head inside a patience ring at 100 %.
0.5 s	Player reads the dish (for example Salmon Nigiri) and taps the Rice station. The rice ball model appears on the plate with a pop, a soft "tap" sound plays, Panda plays <code>Chop_Loop</code> for 0.4 s.
1.2 s	Player taps the Salmon station. The salmon slice model lands on the rice. The plate glows green for 0.3 s because the ingredient set now matches a known dish, and the plate model swaps to the finished dish (<code>Food_SalmonNigiri</code>).
1.8 s	Player taps the customer. The dish flies from the plate to the customer (0.35 s arc), the ring empties into a coin burst, the customer plays <code>Yes</code> then <code>Sitting_Eating</code> . Score, tip and streak update on the HUD with a count-up.
4.3 s	The customer plays <code>Sitting_End</code> , stands and walks out through the door (2.0 s). The stool is free again.

Two to four customers are seated at once from Day 2 on, so the player is always choosing which order to build next. The plate holds exactly one dish in progress; the bin (the sink, `Environment_Counter_Sink`) clears it.

3.2 What makes it repeatable

- **Streak.** Each correct serve with ≥ 50 % patience left adds 1 to the streak; the streak multiplies tips (section 6.5). A wrong dish or an angry customer resets it. Chasing a full-day streak is the expert goal.
 - **Star rating per day** (section 6.6) shown on the day-select screen.
 - **Fixed customer schedule per day.** Orders are seeded per day so a replay is the same puzzle, which makes improvement measurable.
 - **Unlocks.** New dishes and new ingredient stations arrive each day (section 11).
-

4. Controls

Single pointer only. Touch and mouse are handled by the same code path

(`InputEventScreenTouch` / `InputEventMouseButton` pressed events, with the project setting *emulate touch from mouse on*).

Input	Target	Effect
Tap	Ingredient station (any of the 9 tappable models on the counter)	Adds that ingredient to the plate if the plate has fewer than 3 ingredients and the ingredient is not already on it. Otherwise the station shakes (0.2 s) and plays the "no" sound.
Tap	Seated customer (its body or its order bubble)	If the plate holds a completed dish: serves it (correct or wrong, see 6.3). If the plate is empty or incomplete: the customer's bubble bounces (a reminder), nothing else.
Tap	Sink (bin)	Clears the plate. Ingredients splash into the sink with a particle burst. Free, no penalty.
Tap	Pause button (HUD, top-right, 96 × 96 px)	Opens the pause overlay.
Tap	Any menu button	Standard button behaviour with press-squash animation and click sound.

Taps are resolved by a ray from the active camera through the tap position against `Area3D` tap targets on layer 8 ("tap"). Each tappable object owns one `Area3D` named `TapTarget` with a generous shape (at least 1.6 units across) so fingers hit reliably. The topmost hit along the ray wins.

Keyboard (development convenience, not a feature): `Escape` pauses. Nothing else.

5. Camera and perspective

- **Type:** fixed `Camera3D`, no player control, one camera per scene.
 - **Restaurant scene:** position (0, 6.4, 14.5), looking at (0, 1.6, 0): elevation $\approx 18.5^\circ$, FOV 42° vertical, perspective. This reproduces the framing of the pack's `Preview.jpg`: counter across the middle, stools in front, kitchen wall behind, both side walls visible. On a 4:3 iPad the frame spans 22 units of counter width at the counter's depth; on a 16:9 Mac window it spans 29 units (stretch mode `canvas_items`, aspect `expand`, base 1920 × 1440).
 - **Order-bubble camera lock:** none. Bubbles are `Node3D` children of the customer, billboarded toward the camera.
 - **Day start:** the camera starts at (0, 8.0, 19.0) and glides to its position over 1.2 s (ease out) while the door opens.
 - **Shake:** 0.25 s, 0.12 units, on an angry customer leaving.
 - **Menu scene:** camera at (6, 3.2, 10), looking at (0, 1.5, 0), slowly orbiting $\pm 6^\circ$ over 12 s.
-

6. Systems

6.1 Restaurant layout

Everything lives on the 4-unit grid. Floor is 6 tiles wide $\times$ 3 tiles deep (24 $\times$ 12 units):

Floor_Wood for the dining side (front two rows), Floor_Kitchen1 for the kitchen row. The back wall is 6 segments: Wall_Shelves, Wall_Normal, Wall_Shoji, Wall_Shoji, Wall_Normal, Wall_Shelves. Side walls are 3 segments each of Wall_Shoji with Wall_Door as the front-left segment (the customer door). The corner posts are Environment_WoodenBeam.

The counter runs across the room at $Z = 0$: from left to right Environment_Counter_End (mirrored), Environment_Counter_Straight, Environment_Counter_Straight_2, Environment_Counter_Sink, Environment_Counter_Straight, Environment_Counter_End. Four Environment_Stool at $X = -6, -2, 2, 6$ on $Z = 2.2$ are the seats. Kitchen furniture behind the chef against the back wall: Environment_Fridge at $X = 9$, Environment_Oven at $X = 5$, Environment_Cabinet_Shelves at $X = 0$, Environment_CanFridge at $X = -6$, Environment_Cabinet_Doors at $X = -10$. Decoration: two Decoration_Light lanterns hanging front-left and front-right, Decoration_Sign over the door, Decoration_Painting on the back wall, Decoration_Bamboo and Decoration_SakuraFlower on the counter ends, Decoration_Plant2 in the front corners, Decoration_Fish on the right wall, Decoration_Carpet in front of the door. Truck is parked outside the door, visible through the door frame.

6.2 Stations and the plate

Nine stations sit on the counter top ($Y = 1.92$) in a row on the chef's side ($Z = -0.55$), spaced 2.1 units apart from $X = -8.4$ to $X = 8.4$, each a small model on an

Environment_WoodenBoard:

#	Station	Model on the counter	Ingredient added
1	Rice	FoodIngredient_Rice	rice
2	Nori	FoodIngredient_Nori	nori
3	Salmon	FoodIngredient_Salmon	salmon
4	Tuna	FoodIngredient_Tuna	tuna

#	Station	Model on the counter	Ingredient added
5	Ebi	FoodIngredient_Ebi	ebi
6	Tamago	UFP: Egg_Fried	egg
7	Octopus	FoodIngredient_Tentacle	tentacle
8	Cucumber	FoodIngredient_SlicedCucumber	cucumber
9	Sea urchin	FoodIngredient_SeaUrchinOpen	urchin

Stations that are not yet unlocked (section 11) are present but covered by a `Environment_Bowl` turned upside down and do not respond to taps.

The plate is `Environment_Plate` on `Environment_CuttingTable` at the counter centre, $Z = 0.4$ (customer side of the stations, so it reads as "the pass"). It shows the ingredients added so far as their models stacked with a 0.12 unit vertical offset each, and swaps to the finished dish model the moment the ingredient set equals a recipe.

Recipes (ingredient sets are unordered; a dish is complete when the plate's set equals a recipe exactly):

Dish	Finished model	Ingredients	Price
Salmon Nigiri	<code>Food_SalmonNigiri</code>	rice, salmon	8
Maguro Nigiri	<code>Food_MaguroNigiri</code>	rice, tuna	8
Ebi Nigiri	<code>Food_EbiNigiri</code>	rice, ebi	9
Tamago Nigiri	<code>Food_TamagoNigiri</code>	rice, egg	7
Octopus Nigiri	<code>Food_OctopusNigiri</code>	rice, tentacle	10
Onigiri	<code>Food_Onigiri</code>	rice, nori	6
Cucumber Roll	<code>Food_Roll</code>	rice, nori, cucumber	11
Salmon Roll	<code>Food_SalmonRoll</code>	rice, nori, salmon	12
Sea Urchin Roll	<code>Food_SeaUrchinRoll</code>	rice, nori, urchin	14

A plate with 3 ingredients that match no recipe is "ruined": it turns grey (modulate 0.55) and only the sink clears it. A plate with 1–2 ingredients that are a subset of some recipe stays white.

6.3 Customers

Seven rabbit models are used as customers: `Rabbit_Bald`, `Rabbit_Blond`, `Rabbit_Cyan`, `Rabbit_Green`, `Rabbit_Grey`, `Rabbit_Pink`, `Rabbit_Purple` (from `Characters/Normal/glTF`), chosen round-robin per day so no two seated customers share a model. All characters are scaled 0.7.

State machine (one `Customer` scene, script `customer.gd`):

State	Animation (clip names from the glTF)	Duration / exit
Entering	<code>Walk</code>	Moves from the door (−13, 0, 6) to a point 1.2 units in front of its stool at 3.0 u/s, turns to face the counter.
Sitting down	<code>Sitting_Start</code>	Clip length (≈ 1.0 s). Order bubble appears at the end.
Waiting	<code>Sitting_Idle</code>	Patience drains from 100 % to 0 % over <code>patience_seconds</code> (section 11). At 30 % the ring turns orange and the customer plays <code>No</code> once every 4 s.
Served correct	<code>Yes</code> then <code>Sitting_Eating</code>	<code>Yes</code> (≈ 1.0 s) then eating 2.5 s. Pays (6.5).
Served wrong	<code>No</code>	Patience −25 points, bubble bounces, the dish is thrown to the sink. Back to Waiting.
Angry	<code>No</code> then <code>Sitting_End</code> , <code>Walk</code>	Patience reached 0. Strike +1. Leaves the way it came at 4.0 u/s.
Leaving happy	<code>Sitting_End</code> , <code>Walk</code>	Walks back to the door at 3.0 u/s, is freed at the door.

Customers never overlap: the spawner only sends a customer when a stool is free and at least `spawn_gap` seconds passed since the previous spawn.

Order bubble (`OrderBubble` node inside `Customer`): a `Node3D` at 3.4 units above the stool holding the dish model at scale 1.6, rotating 60°/s, plus a patience ring: a `MeshInstance3D` quad (1.4×1.4) with a shader that draws an arc from 0 to `patience` (0–1) in green > 0.6 , yellow > 0.3 , red otherwise, always facing the camera.

6.4 The chef

Panda from Characters/With Knife and Pan/gLTF behind the counter at (0, 0, -1.9), facing the customers, scale 0.7. Animations used: Idle (loop), Chop_Loop (played 0.4 s on every ingredient tap, blended 0.1 s), Yes (correct serve), No (wrong serve or clearing a ruined plate), Wave (day start and day end), Idle_Holding (unused), HitReact (on an angry leave). The chef never moves.

6.5 Scoring

- **Dish price** as in the recipe table.
- **Tip:** $\text{price} \times 0.5 \times \text{patience_left}$ (patience_left in 0–1), rounded down.
- **Streak:** consecutive correct serves with $\text{patience_left} \geq 0.5$. Tip is multiplied by $1 + 0.25 \times \min(\text{streak}, 8)$ (max $\times 3$). Displayed on the HUD as "x1.25", "x1.5" ... with a pop each time it grows. Reset to 0 on a wrong serve or an angry leave.
- **Day earnings** = sum of prices + tips. **Target earnings** per day in section 11.
- **Happy customers** = served correctly. **Angry customers** = left at 0 patience.

6.6 Day rating

Stars	Condition
★	Day finished (all customers of the schedule handled) with fewer than 3 strikes.
★★	★ and happy ≥ 80 % of customers.
★★★	★★ and 0 angry customers and earnings $\geq$ target.

6.7 Fail and win states

- **Strikes.** Each angry customer is one strike. Three strikes end the day immediately: the remaining customers are cancelled, the "Closed early" results screen shows, the day is not marked complete and can be retried.
- **Day complete** when every customer of the schedule has left (happy or angry) and strikes < 3 .
- **Game win:** completing Day 5. The ending screen (10.6) plays, all days stay replayable.

6.8 Persistence

user://save.json: per day {completed, stars, best_earnings, best_streak}, plus music and sfx booleans. Written on every results screen and every settings change.

7. Scene and node architecture

All scenes are created in the Xogot editor. Node names are the ones listed here.

Scene	Root	Owns
<code>res://scenes/main.tscn</code>	Main (Node)	SceneSlot (Node) holding the current screen; Fader (CanvasLayer, layer 100) with Rect (ColorRect, iris-fade shader). Script <code>main.gd</code> : <code>switch_to(path)</code> .
<code>res://scenes/ui/main_menu.tscn</code>	MainMenu (Node3D)	Environment (WorldEnvironment), Sun (DirectionalLight3D), Camera (Camera3D), Diorama (Node3D: one counter segment, two stools, Panda idle, a Rabbit_Pink sitting, three dishes on the counter, one lantern), UI (CanvasLayer: Title, Subtitle, Play, Stats, Credit).
<code>res://scenes/ui/day_select.tscn</code>	DaySelect (Control)	Background, Title, Back, Cards (HBoxContainer with five DayCard instances).
<code>res://scenes/ui/day_card.tscn</code>	DayCard (Button)	Layout (VBoxContainer): DayLabel, NameLabel, Stars (HBoxContainer of 3 TextureRect), Best.

Scene	Root	Owns
<code>res://scenes/restaurant.tscn</code>	Restaurant (Node3D), script <code>restaurant.gd</code> (day controller)	Environment, Sun, FillLight (OmniLight3D, warm, over the counter), Camera, Room (Node3D: floor tiles, walls, beams, decoration — all static instances), Kitchen (Node3D: fridge, oven, cabinets), Counter (Node3D: the six counter segments), Stations (Node3D: nine Station instances named StationRice ... StationUrchin), Plate (Plate instance), Sink (Node3D with TapTarget), Chef (Chef instance), Seats (Node3D with Seat1 ... Seat4, each a Marker3D over its stool), Door (Marker3D), Customers (Node3D, runtime parent for customer instances), Spawner (Node, script <code>spawner.gd</code>), HUD (HUD instance).
<code>res://scenes/actors/station.tscn</code>	Station (Node3D), script <code>station.gd</code>	Board (Environment_WoodenBoard), Model (Node3D, the ingredient model is set per instance), Cover (Environment_Bowl, hidden when unlocked), Label (Label3D), TapTarget (Area3D + CollisionShape3D box 1.8 × 1.2 × 1.6). Exported: <code>ingredient: String</code> .
<code>res://scenes/actors/plate.tscn</code>	Plate (Node3D), script <code>plate.gd</code>	Table (Environment_CuttingTable), Dish (Environment_Plate), Stack (Node3D, ingredient models are instanced here), Finished (Node3D, the dish model), Glow (OmniLight3D), TapTarget.
<code>res://scenes/actors/chef.tscn</code>	Chef (Node3D), script <code>chef.gd</code>	Model (Panda with knife and pan).

Scene	Root	Owns
<code>res://scenes/actors/customer.tscn</code>	<code>Customer</code> (<code>CharacterBody3D</code>), script <code>customer.gd</code>	<code>Model</code> (<code>Node3D</code> ; the rabbit model is instantiated under it at spawn), <code>OrderBubble</code> (<code>Node3D</code> : <code>DishHolder</code> , <code>Ring</code>), <code>TapTarget</code> (<code>Area3D</code> , capsule 1.6×3.0), <code>Shape</code> (<code>CollisionShape3D</code>).
<code>res://scenes/ui/hud.tscn</code>	<code>HUD</code> (<code>CanvasLayer</code>), script <code>hud.gd</code>	<code>Top</code> (<code>Control</code>): <code>Earnings</code> , <code>Streak</code> , <code>Strikes</code> (3 <code>TextureRect</code>), <code>Customers</code> ("3 / 12"), <code>DayLabel</code> ; <code>PauseButton</code> ; <code>Message</code> (<code>Label</code> , centre pops); <code>PauseMenu</code> (<code>Control</code> : <code>Dim</code> , <code>Panel</code> with <code>Resume</code> , <code>Restart</code> , <code>Days</code> , <code>Music</code> , <code>Sound</code>).
<code>res://scenes/ui/results.tscn</code>	<code>Results</code> (<code>Control</code>), script <code>results.gd</code>	<code>Background</code> , <code>Panel</code> : <code>Title</code> , <code>Stars</code> , <code>Stats</code> , <code>Buttons</code> (<code>Next</code> , <code>Replay</code> , <code>Days</code>).
<code>res://scenes/ui/ending.tscn</code>	<code>Ending</code> (<code>Node3D</code>), script <code>ending.gd</code>	The restaurant diorama with <code>Panda</code> playing <code>Wave</code> and four rabbits <code>Sitting_Eating</code> , UI with <code>Title</code> , <code>Stats</code> , <code>Back</code> .
<code>res://scenes/fx/burst.tscn</code>	<code>Burst</code> (<code>GPUParticles3D</code>)	One-shot burst reused for coins, splashes and confetti (colour set per use).

Autoloads: `Game` (`res://scripts/autoload/game.gd`): day list, save data, scene switching, results hand-off. `Audio` (`res://scripts/autoload/audio.gd`): SFX pool and music.

Signals (up) and calls (down).

- `Station.tapped(ingredient)` → `Restaurant.on_station_tapped` → `Plate.add(ingredient)`.
- `Plate.changed(ingredients, dish_or_empty)`.
- `Customer.tapped(customer)` → `Restaurant.on_customer_tapped` → compares `Plate.dish` with `customer.order`.
- `Customer.served(correct)`, `Customer.left(happy: bool)` → `Restaurant` updates score, strikes, HUD, streak.

- `Spawner.spawn_requested(order, seat)` → `Restaurant` instances `customer.tscn` under `Customers`.
- `Restaurant.day_finished(result: Dictionary)` → `Game.day_complete(result)` → results screen.

Collision layers: 1 world (unused by gameplay), 8 tap targets. Physics is only used for the tap raycast; customers move by tween, not by physics.

8. Art direction

Look. Warm wooden izakaya, exactly the palette of the pack's `Sushi_Atlas.png` (512 × 512, one atlas for every model, so the whole scene is one material). Lighting:

`DirectionalLight3D` at elevation 42°, azimuth -25°, colour (1.0, 0.93, 0.82), energy 1.2, shadows on; one warm `OmniLight3D` (1.0, 0.8, 0.55) energy 1.6, range 14 above the counter; ambient from a flat colour (0.62, 0.55, 0.5) energy 0.6. Environment: background is a flat colour (0.18, 0.13, 0.12) (the room fills the frame; no skybox is needed); tonemap Filmic, white 6; glow on (intensity 0.4) so the order rings and the plate glow read; SSAO off (mobile).

Models by role (all Sushi Restaurant Kit unless marked UFP):

- Structure: `Floor_Wood`, `Floor_Kitchen1`, `Wall_Shelves`, `Wall_Normal`, `Wall_Shoji`, `Wall_Door`, `Environment_WoodenBeam`.
- Counter: `Environment_Counter_End`, `Environment_Counter_Straight`, `Environment_Counter_Straight_2`, `Environment_Counter_Sink`, `Environment_CuttingTable`, `Environment_Plate`, `Environment_WoodenBoard`, `Environment_Bowl`.
- Seats: `Environment_Stool`.
- Kitchen: `Environment_Fridge`, `Environment_Oven`, `Environment_Cabinet_Shelves`, `Environment_CanFridge`, `Environment_Cabinet_Doors`, `Environment_Pot_1_Filled`, `Environment_ChukamanSteamer`, `Environment_KitchenKnives`, `Environment_Bottles`.
- Decoration: `Decoration_Light`, `Decoration_Sign`, `Decoration_Painting`, `Decoration_Bamboo`, `Decoration_SakuraFlower`, `Decoration_Plant2`, `Decoration_Fish`, `Decoration_Carpet`, `Truck` (outside).
- Ingredients: `FoodIngredient_Rice`, `FoodIngredient_Nori`, `FoodIngredient_Salmon`, `FoodIngredient_Tuna`, `FoodIngredient_Ebi`, `FoodIngredient_Tentacle`, `FoodIngredient_SlicedCucumber`, `FoodIngredient_SeaUrchinOpen`, **UFP:** `Egg_Fried`.
- Dishes: `Food_SalmonNigiri`, `Food_MaguroNigiri`, `Food_EbiNigiri`, `Food_TamagoNigiri`, `Food_OctopusNigiri`, `Food_Onigiri`, `Food_Roll`, `Food_SalmonRoll`, `Food_SeaUrchinRoll`; on the menu diorama also `Food_Ramen`, `Food_Gyoza`, `Food_Dango`.
- Characters: `Panda` (with knife and pan), the seven `Rabbit_*` (normal).

Feedback effects (no pack assets needed): ingredient pop-in (scale 0 → 1.15 → 1 over 0.25 s), dish fly-to-customer tween, coin burst (gold particles) on pay, splash burst (blue-white) at

the sink, plate glow light pulse when a recipe completes, patience ring shader, iris fade between screens, HUD count-ups and pops, camera shake on strikes.

Unused files. Both packs contain many more models (the Ultimate Food Pack's burgers, pizzas, desserts; the kit's `Environment_Sofa`, `Environment_Table`, `Environment_ToriiGate`, all `Blends`, `FBX` and `OBJ` duplicates of the glTF files). They stay in the project untouched; the Blend folders carry a `.gdignore` so the editor does not try to import them (that needs Blender).

9. Audio plan

Neither pack contains audio. All sounds are synthesized WAV files generated by a script in `tools/` and committed under `res://assets/audio/` (section 13). Triggers:

Event	Sound	Notes
Station tap	short wooden "tok", 3 pitch variants	plus <code>Chop_Loop</code> on the chef
Recipe complete	two-note ascending chime	
Wrong combination (ruined plate)	low buzz	
Serve (dish flies)	whoosh	
Correct serve / pay	coin jingle, pitch rises with streak	up to 8 steps
Wrong serve	"no" buzz + customer <code>No</code>	
Sink	splash	
Customer enters	door bell (<code>Decoration_Bell</code> is the visual reference on the wall)	
Patience low (< 30 %)	soft tick every 1 s for that customer	max one ticking customer at a time
Angry leave	descending three notes + camera shake	
Day start	wave chime	
Day complete	short fanfare	
Closed early	descending sting	
UI tap	click	
Music: menu	calm koto-like plucked loop, 32 s	
Music: days 1–3	upbeat loop, 24 s	
Music: days 4–5	same loop at +6 BPM	
Music: results	5 s sting	

Buses: Master, Music, SFX. Pause menu toggles mute per bus. Music ducks -8 dB during the results fanfare.

10. UI and HUD

Base resolution 1920 × 1440, `canvas_items` stretch, aspect `expand`. Default font with outlines (no font file in the packs, section 13). Body 36 px, titles 84–150 px, all with a dark outline for contrast over the warm scene.

10.1 Main menu

Title "SUSHI" (150 px, slight idle wobble $\pm 2^\circ$), subtitle "tap the ingredients · serve the rabbits · keep the streak", Play button (420 × 130 px), stats line "Best day: 312 coins · 9 / 15 stars", credit "Models by Quaternius (CC0) · quaternius.com". Background: the live diorama.

10.2 Day select

Five cards (300 × 360 px) in a row: "Day 1" ... "Day 5", the day name, three star slots, best earnings. Locked days show a dimmed card with a lock glyph drawn as a rounded rectangle and arc (no lock texture exists). Back button top-left.

10.3 HUD (in a day)

- Top-left: earnings "¢ 128" with the coin count-up, streak badge "×1.5" that pops when it changes.
- Top-centre: "Day 2 · 7 / 10" customers handled.
- Top-right: three strike marks (dim circles that fill red) and the pause button.
- Centre: transient messages ("Perfect!", "+12 tip", "Wrong dish", "Closed early") that pop and fade in 1.0 s.
- No ingredient buttons: stations are in the world (pillar 2).

10.4 Pause

Dim overlay, panel with Resume, Restart day, Day select, Music on/off, Sound on/off.

10.5 Results

Title "Day 2 complete" or "Closed early", three star slots animating in one by one, stats: earnings vs target, happy / total, best streak, angry count, time. Buttons: Next day (or Finish on day 5), Replay, Days.

10.6 Ending

Panda waves behind the counter, four rabbits eat, confetti bursts; title "Thanks for playing", totals (coins, stars), Back to days.

11. Content plan: the five days

Day	Name	Customers	Seats in use	Dishes available (cumulative)	New stations	Patience (s)	Spawn gap (s)	Target earnings
1	Opening	8	2	Salmon Nigiri, Maguro Nigiri, Onigiri	Rice, Nori, Salmon, Tuna	30	6	60
2	Regulars	10	3	+ Ebi Nigiri, Cucumber Roll	Ebi, Cucumber	26	5	100
3	Lunch rush	12	4	+ Tamago Nigiri, Salmon Roll	Tamago	22	4	150
4	Festival	14	4	+ Octopus Nigiri, Sea Urchin Roll	Octopus, Sea urchin	20	3.5	210
5	Grand finale	16	4	all nine	—	18	3	280

Rules: the order list per day is generated from a fixed seed ($1000 + \text{day}$) drawing uniformly from that day's available dishes, with the constraint that the same dish never appears three times in a row. From Day 3, every fourth customer orders two dishes in sequence (the second bubble appears after the first is served, with a fresh patience timer). Spawn gap is the minimum time between two entrances; a customer also needs a free seat.

Tutorial: Day 1 shows three timed messages in the HUD centre: "Tap Rice, then Salmon" (at first customer), "Tap the rabbit to serve" (when a recipe completes the first time), "Tap the sink to start over" (the first time a wrong ingredient is added). They never show again once Day 1 is completed.

12. Scope

12.1 Cut list — deliberately OUT of v1

- Drinks, soups and cooked dishes (`Food_Ramen` , `Food_Udon` , `Food_Gyoza` , `Food_Chukaman` , `Food_Dango` , the oven, pots and steamer are decoration only).
- Moving the chef; the chef stays behind the pass.
- Multiple plates / holding several dishes.
- Table service (`Environment_Table` , `Environment_Chair1/2` , `Environment_Sofa` unused).
- Upgrades, money spending, shop.
- Leaderboards, Game Center, cloud save.
- Ingredient chopping mini-games or timing bars.
- Character customisation.
- Localisation beyond English.
- Landscape/portrait switching: landscape only.

12.2 Build order (each step ends with a run in Xogot, a screenshot check, and a commit)

1. Project settings: name, main scene, 1920 × 1440 `canvas_items` expand, landscape, emulate touch from mouse, tap layer name, `pause` input action. Autoloads `Game` , `Audio` .
2. Audio generation script and the WAV set.
3. `main.tscn` with the fader; `main_menu.tscn` with a first pass of the diorama; run: menu visible.
4. `restaurant.tscn` static room: floor, walls, counter, stools, kitchen, decoration, lights, camera; run: the Preview.jpg framing is reproduced.
5. `station.tscn` + `plate.tscn` + recipes; tapping builds dishes; run.
6. `chef.tscn` with animations reacting to taps.
7. `customer.tscn` : walk in, sit, bubble with dish and ring, patience, leave; `spawner.gd` with the Day 1 schedule; serving logic, scoring, strikes, HUD.
8. Day schedule table for all five days, unlock covers, results screen, save file, day select, pause.
9. Ending screen.

0. Feedback pass: bursts, tweens, glow, shake, messages, music/SFX hookup, tutorial messages.
 1. Full playthrough of Day 1 to Day 5 watched through screenshots; tuning of patience and spawn gaps.
 2. Final commit and push.
-

13. Missing assets and open questions

Need	Used by	Resolution
Audio (all of section 9)	everything	Synthesized WAVs committed to <code>res://assets/audio/</code> , generated by <code>tools/gen_audio.py</code> .
UI font	all screens	Godot's default font with outlines.
2D icons (coin, star, strike, lock, pause)	HUD, cards, results	Drawn in code with <code>_draw()</code> primitives (circles, stars as polygons) — no textures needed.
Patience ring	order bubble	A quad with a small shader; no texture needed.
Speech-bubble backdrop	order bubble	The dish floats inside the ring; no backdrop.
Tamago (egg) ingredient	Tamago Nigiri	UFP: <code>Egg_Fried</code> (the sushi kit has no egg).
Door open animation	customer entrance	<code>Wall_Door</code> is static; the customer simply walks through the opening.
Coin model	pay feedback	Particle burst in gold; no coin mesh.

Open questions for Marco (answered with the assumption in brackets): none that block the build. Assumptions: landscape only; one save slot; touch and mouse identical; the Ultimate Food Pack is used only for the egg and as menu-diorama dressing.

End of document.